WHAT EVERYONE SHOULD KNOW ABOUT SCHEDULING

BÖHNLEIN, T., DE VITA, M., MATZOROS, C. K., PAPP, P. A., STEINER, R. S., AND YZELMAN, A. N.

Abstract. Scheduling tasks on compute systems of any scale has become ubiqutous, and is no longer a specialist topic in computer science and applied mathematics. Over the past few years, Huawei has endeavoured to further develop the theoretical foundations of scheduling, crucially from the perspective that in modern compute systems, data movement is by far more costly than actual computation. This industry-wide trend has wide-ranging practical implications– not only in areas ranging from parallel system to algorithm design, but also on software, communication layers, and run-time systems. The article before you summarises our recent advances, outlines their practical implications, and translates these to concrete advice on parallel system and software design.

1. Introduction

scheduling is everywhere: CPUs, NPUs, SOCs
scheduling becomes more intricate at scale:

- multi-die,
- multi-device,
- cloud & supercomputer scale

computation as a (Hyper)DAG
scheduling is not partitioning

1.1. **Computation model.**

PRAM is not sufficient, must cost communication
note that ideally, furthermore: hierarchical

- hierarchical processor designs, hierarchical nodes, hierarchical networks

latencies matter (can cite Anastasiadis if ready)

Date: 4th May 2026.

All work was done while the authors were affiliated with the Computing Systems Lab, Huawei Research Zürich. Authors Pál András Papp and Albert-Jan N. Yzelman have left Huawei at the time of publication.

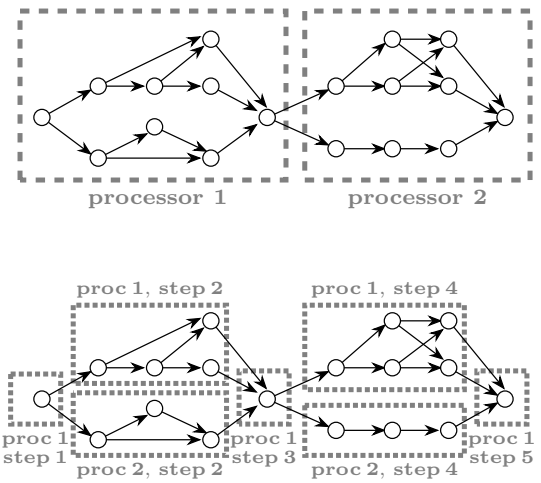


Figure 1.1. Here's a picture for "scheduling is not partitioning", if needed. Old description is in comments.

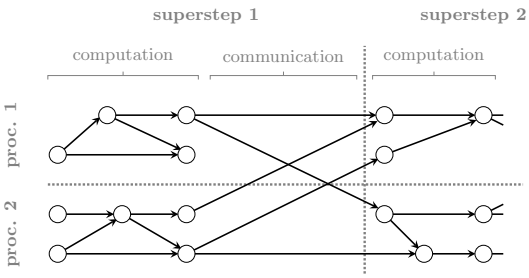


FIGURE 1.2. Here's an example DAG-BSP-schedule picture, if needed. Old descriptions in comments.

1.2. **Applications.** a summary of where scheduling is applied at present

- run-time systems, such as OpenMP;
- communication layers, such as MPI, GASNET, others (PRAM/PGAS?);
- within compilers (to various degrees)
- offline system design (e.g., Wireless)
- not meant as an exhaustive list of applications, but rather to index the quite different domains of applicability

2. Similar problems, similar results, and solutions

Partitioning, streaming, imply the same work-arounds.

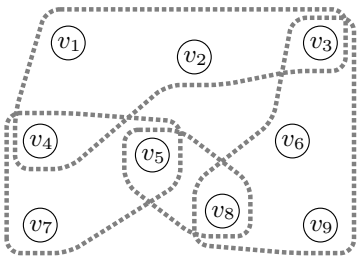


FIGURE 2.1. Here's an example hypergraph partitioning picture, if needed. Old description in comments.

3. Scheduling is hard

The complexity of problems is computer science is often defined via the concept of *NP-hardness*. Intuitively speaking, an NP-hard problem is one that is very unlikely to have an algorithm that can always find the optimal solution efficiently (i.e., with a running time that is polynomial in the size of the problem). In particular, if there was an efficient algorithm for any of the NP-hard problems, then all the other NP-hard problems could be solved efficiently as well, by reducing these problems to each other. Thousands of problems are known to be NP-hard, and many of these have been extensively studied by the community for decades; as such, when a problem is NP-hard, it is generally accepted that no polynomial-time algorithm exists for it.

Besides NP-hardness, there are other concepts that show that the problem is even more challenging. Inapproximability, for instance, means that even finding a solution that is close to the optimal value is NP-hard. "Close to optimal" here means a solution that is a constant multiplicative or additive factor away from optimal. Parameterized complexity, on the other hand, studies whether the problem remains hard if some parameters of the input are fixed to a small value. Here, the crux lies with exposing a parameter that is suitably constrained in practical applications.

3.1. **Horizontal data movement.** Through these concepts, one may demonstrate the hardness of scheduling even in very simple cases: with unit execution time for all tasks and unit communication delays between the processors, finding the optimal schedule is NP-hard [RS87]. In more realistic models such as BSP, the problem is even more difficult: the optimal solution is already NP-hard for very special classes of DAGs, such as two-layer DAGs or in-trees where each vertex has at most one outgoing edge. Figure 3.1 illustrates both DAG structures. Moreover, in general DAGs, the optimal cost is even NP-hard to efficiently approximate beyond a specific ratio $(1 + \delta)$, for some $\delta > 0$; in technical terms, the problem does not allow for a so-called Polynomial-Time Approximation Scheme, and the problem is *APX-hard*. Even if the assignment of nodes to processors and supersteps is already fixed, the subproblem of scheduling only the necessary communication is already NP-hard [PAY25].

A classic notion in parallel computing is that of overhead: the difference between the computational resources used versus simply running a sequential algorithm to solve the problem, or, more formally,

$$O(n, p) = pT_p(n) - T_{\text{seq}}(n).$$

Here, $T_{\text{seq}}(n)$ represents the cost (e.g., time) of a sequential algorithm to solve the given problem of some size n, while $T_p(n)$ represents the cost of parallel solution when using p processors [GGK02]. We adapt this concept to define an *surplus cost* as $O(n, p)/p = T_p(n) - T_{\text{seq}}(n)/p$; or, intuitively, the cost of parallelisation subtracted its minimal unavoidable cost. For general DAGs, the optimal surplus cost cannot be approximated within polynomial time to a multiplicative or an additive factor almost linear[1] to the problem size n– meaning, any polynomial scheduling algorithm for general DAGs can be an almost arbitrary factor away from optimal. This hardness result restricts itself to the case T_p and T_{seq} are derived from the same underlying DAG and holds both with and without recomputation [PAY25, PBY26].

[1] to be precise, a factor is within $n^{1/(\log \log n)^{\delta}}$, for some $\delta > 0$

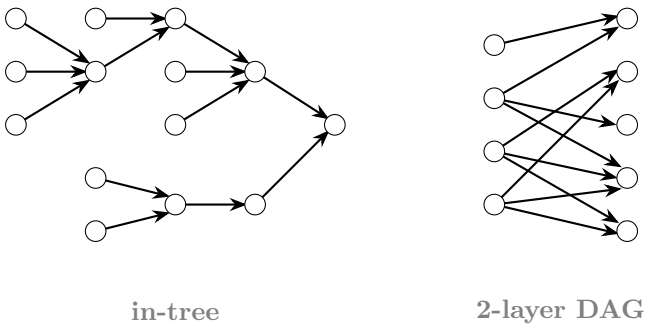


Figure 3.1. Illustration of special subclasses of DAGs: in-trees and 2-layer DAGs. Scheduling in the BSP model is proven to be already hard on these rather simple DAG classes. In contrast to this, in simpler scheduling models, the problem is still efficiently solvable for these particular cases.

3.2. **Vertical data movement.** BSP only considers the cost of moving data horizontally from the memory associated with one processor to memory associated with another. However, when executing real computations – even those that employ just a single processor – one of the most important factors is that fast memory in the system (e.g. cache) often has very limited capacity, whereas we also have some other memory (e.g. RAM) with much larger capacity but drastically slower latency and throughput. In practice, often the cost of moving data from slow memory to fast memory (or vice versa) is the bottleneck of executing practical computations. Thus, another vital aspect of scheduling computations is vertical data movement, otherwise known as I/O cost.

The *red-blue pebble game* of Hong and Kung [HK81] serves as a basis for analysing I/O cost. This game represents a single-processor scheduling problem in a two-level memory hierarchy (with e.g. cache and RAM), where the goal is to minimize the I/O operations while executing the computations. Using it, researchers have analysed the I/O cost of a large variety of algorithms and developed lower bounds for many important computations, such as matrix multiplication, fast Fourier transform or attention in neural networks [HK81, SY24]. We discuss these algorithm-specific approaches further in Section 4; for general DAGs, however, finding the optimal I/O cost is again a problem that is NP-hard to approximate to any reasonable factor [BPY25].

3.3. **Combined horizontal & vertical data movement.** For a more realistic model of parallel computation, the BSP model that captures multiple processors and horizontal communication should be combined with the red-blue pebble game that captures memory limitations and I/O costs. Our work generalises the red-blue pebble game to a parallel setting with unit data movement costs (pebbling) and with realistic data movement costs g, l (BSP), resulting in the multi-processor red-blue pebbling model (MPP), respectively, the more complex Multi-BSP model [Val11]. Furthermore, modern memory subsystems are increasingly hierarchical: increasingly deep and wide. Savage extended the Hong and Kung to multiple levels [Sav95], subsumed by MPP and Multi-BSP that naturally also apply to deeper NUMA hierarchies [Val11, BPY25, PBY25]. We denote by depth $d = 1$ the basic pebbling case of having both a fast and slow memory, while every added layer in the memory hierarchy increases d.

The hardness of both BSP scheduling and red-blue pebbling carry over to these combined models: if the special cases of $p = 1$ and $d = 1$ are NP-hard and inapproximable in polynomial time, then the general cases for p, d are as well. Nevertheless, some notion of relative hardness may be established: one can show a worst-case example where the two-stage schedule is an arbitrary factor $\Omega(n)$ away from optimal, even if both stages are optimal separately [BPY25]. Such results can inspire algorithmic approaches for scheduling over NUMA systems– for example, Section 4.3 confirms that holistic scheduling can indeed outperform two-stage scheduling, a result that should inspire modern software design.

3.4. **Partial computations.** The original red-blue pebble game assumes that every operation is computed in one go, requiring all of its inputs in fast memory simultaneously. However, if the operation is e.g. the addition or multiplication of numerous inputs, then it can also be executed in multiple steps: we first aggregate only some of the input values, maybe even save this partial results into slow memory to load it back later, and then continue with another subset of the inputs. Such a partial computing strategy might allow for a lower I/O cost in some cases, and might even be required to realise a feasible solution if, e.g., the operation has a high number of inputs.

The red-blue pebble game can naturally be extended with such partial computations to allow for a more realistic model of I/O costs [Sob26, PSY25]. In this model, we establish NP-hardness for determining whether the optimum is affected by allowing partial computation or not, show the difference between optimal solutions with and without pebbling can be the full multiplicative factor n, and prove there is no polynomial-time algorithm that approximates the optimum to any useful multiplicative or additive factor [PSY25]. Regardless, we can establish useful lower bounds for specific algorithms and find algorithms that achieve them: the next section details the intuitions and tools, both when partial computations are allowed and not.

3.5. **Realism versus complexity.** While MPP and Multi-BSP –augmented with partial computations or not– offer increasingly realistic models for the scheduling problem, they increase the complexity of finding optimal DAG schedules. Do we absolutely require the increased realism in our models?

For horizontal data movement, we indexed model variations beyond those described in this exposition and related them to additional established models. In particular, we identified a taxonomy that captures not only BSP, but also the Hockney (alpha-beta) and single-port duplex models [Hoc94, ÖBU+19]. Crucially, the work relates how optimal solutions in different models relate within constant factors [PAY25,

Thm. 1]: if absolute performance need not be 100% or can be achieved via later optimisation pass, selecting the simplest model for horizontal communication thus suffices.

However, even the simplest model has no practically useable approximation algorithm for all possible input DAGs, while crucial workloads and complex system architectures mandate the use of the more intricate models described above. Sections 4 to 6 discuss dealing with the hardness of these problems by, in turn, expert humans in the loop, software automation of expert strategies, and sophisticated algorithms that find optimised schedules automatically and despite the established hardness of doing so.

4. Avoiding hardness by avoiding generality

We have shown that scheduling under realistic cost models hard, yet practice does regularly realise software that achieves near-optimal performance on parallel architectures and systems. Positive results –meaning polynomial-time algorithms that find the optimum or approximate it within a useable factor– do indeed exist: for example, polynomial-time solutions exist for DAGs consisting only of independent chains and for a small number of processors [PAY25, BPY25, PSY25, PBY25].

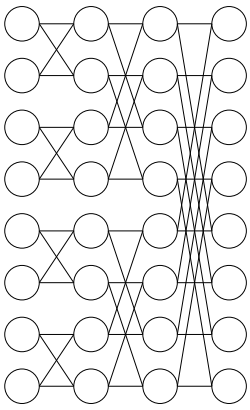
While resulting algorithms remain very inefficient in practice [PAY25], the above example does illustrate how to successfully avoid hardness: by *not* considering scheduling as a general problem, and instead focusing on scheduling specific computations or specific types of computations. This section introduces common approaches along the different types of data movement:

(1) horizontal data movement (communication);
(2) vertical data movement (I/O); and
(3) data movement through NUMA hierarchies (combined model).

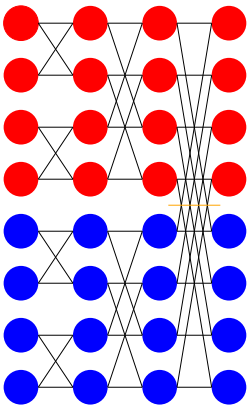
We focus on techniques that prove optimality for specific problems or specific classes of problems for each type. We couple outlining these state-of-the-art methodologies with novel insights from our research– for example, we discuss recursive and hierarchical approaches for dealing with NUMA, implications on run-time system design for modern large-scale computing systems, implications on parallel frameworks for AI and beyond, and the case of irregular computations.

4.1. Horizontal data movement.

Minimisation of horizontal data movement, i.e., communication minimisation, comes naturally when adopting a model of computation that assigns explicit costs to data movement and synchronisation, such as, indeed, BSP [Val90]. This model revolves around supersteps and h-relations that together quantify the cost of data movement– see Section 1.1. Costs are parametrised in the number of processors p and some problem-specific parameter n, such as a problem size. The key phrase here is *problem-specific*: in HPC, by far most known practical parallel algorithms are BSP, and literature is rich with examples of optimal algorithms or algorithms that operate within an optimal trade-off regime. We give two such examples, fast Fourier transformation (FFT) and matrix multiplication, and demonstrate how optimal algorithms and optimal trade-offs may be derived.

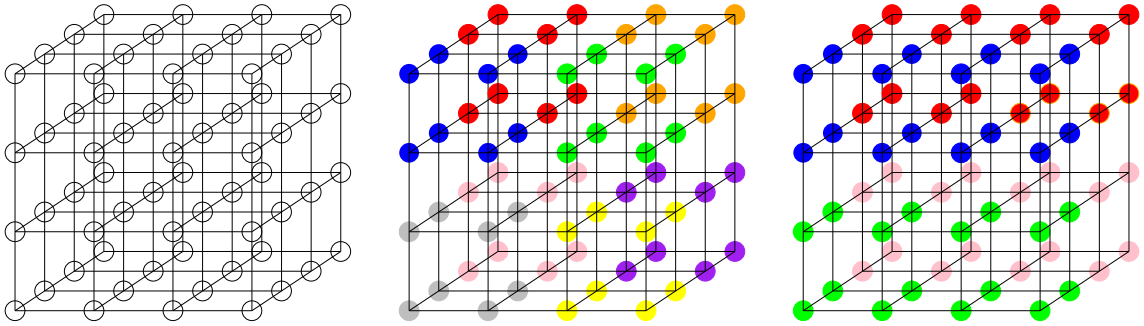


(A) Directed acyclic graph representation of a fast Fourier transform on an input of length eight. Connections should be read from left to right.



(B) A two-processor parallelisation assigns vertices to a blue or red processor. An orange line indicates cut edges that require communication.

Figure 4.1. Butterfly graphs and their application to fast Fourier transforms.



(A) The computational DAG for multiplying two 4×4 matrices. (B) Communication-optimal work distribution over eight processors. (C) Memory-optimal work distribution over four processors.

FIGURE 4.2. Cube graphs of size $4 \times 4 \times 4$ and their application to matrix–matrix multiplication.

Fast Fourier transforms and butterfly DAGs. Consider the visual representation of a one-dimensional FFT of length $n = 8$ depicted in Figure 4.1a. This so-called *butterfly graph* consists of three phases, and of $\log n$ phases for general n. When distributing the computation over multiple processors, different vertices are assigned to different processes. Edges between vertices assigned to different processors induce communication between processes. By cutting the graph horizontally, we visually confirm the computation may proceed in parallel without communication for the first phase up to $p \leq 4$, while the first two phases may be computed concurrently for up to $p \leq 2$ processors. Figure 4.1b visualises the latter, from which we furthermore observe that this particular DAG partitioning results in exactly n cut edges. The communication from phase two to three furthermore is balanced amongst the processors– the communication cost hence is proportional to n/p, while the number of supersteps is one and the processor-local work is proportional to $(n \log n)/p$. These bounds, derived solely by analysing the computational DAG structure, can be achieved by practical algorithms for general n [PY88, Val90, IB01].

Given that the sequential FFT algorithm has a work cost proportional to $n \log n$ –and any computation that maps to a butterfly graph has a work cost that is *at least* proportional to $n \log n$– the total parallelisation cost of $(n \log n)/p$ is optimal[2], and any computation that maps to a butterfly graph can be optimally parallelised by this singular lower-bound analysis and parallelisation recipe. Colloquially, algorithms backed by (or resulting from) such proofs of optimality are known as *immortal algorithms*.

Matrix–matrix multiplication. Let us consider two matrices A, B both of size $n \times n$, and the matrix–matrix multiplication $C = AB$. Figure 4.2a depicts the internal nodes[3] of the resulting DAG for $n = 4$, where, for example, the matrix A could span the horizontal and vertical axes (the front of the cube), the matrix B the horizontal and depth axes (the top of the cube), and the output matrix C the vertical and depth axes (the side of the cube)– in which case the direction of the horizontally-oriented edges would face towards C, while the depth- and vertically-oriented edges would face *away* from A and B, respectively. All vertices shown correspond to multiplications $a_{ik}b_{kj}$ of the canonical triple-loop matrix–matrix multiplication algorithm, which, along the edges towards C, are summed into the output c_{ij}.

In the same vein as for the FFT, we may construct a parallel algorithm by exploiting the geometric features of the underlying cube DAG. While coloring vertices to specific processors, again edges between differently-coloured nodes will induce communication. Therefore the work distribution that minimises communication is one that minimises such cuts, which in the case of a cube means identifying p volumes for which the surface areas (where edge cuts occur) are minimised. Figure 4.2b exemplifies such a distribution, which achieves a communication cost proportional to $(n/p^{1/3})^2 = n^2/p^{2/3}$ in two supersteps: first, every input from A and B are spread out over $p^{1/3}$ processors while second, every output in C is accumulated from partial results of $p^{1/3}$ processors. In-between, a process-local sequential multiplication takes place that costs $(n/p^{1/3})^3 = n^3/p$ work per processor, which is work-optimal as well. The above geometric argument is extensible to a formal proof of optimality[4] in terms of horizontal data movement for general n, and practical algorithms that achieve this bound exist [DNS81, Ber89, ACS90, MT99, ITT04].

[2] assuming the work associated with each vertex has homogeneous cost and that p is large enough compared to n

[3] i.e., the DAG excluding the input and output nodes

[4] again, for large enough n compared to p

Parametrisation and poly-algorithms. The resulting so-called 3D algorithm for matrix multiplication, however, is not an immortal algorithm: while communication- and work-optimal, the processor-local memory requirement is proportional to $n^2/p^{2/3}$– larger than the ideal bound of n^2/p. Subdividing the matrices A, B^T, and C into p square blocks results in the so-called 2D algorithms, which are work- and memory-optimal with costs proportional to n^3/p and n^2/p, respectively. These algorithms are not communication-optimal, however, as processors need to communicate $\sqrt{p}-1$ blocks of size n^2/p over $\sqrt{p}-1$ supersteps to maintain memory optimality. Figure 4.2c depicts the resulting work distribution for $n=p=4$, which visualises a higher surface-to-volume ratio compared to the 3D algorithm, leading to the suboptimal communication cost.

Indeed there exists a discrete trade-off space between optimality in communication and optimality in memory, confirmed by theoretical studies [ITT04]. This identifies an opportunity for programs to, when available, use more memory in order to reduce communication– such decisions could even be made automatically, at run-time. To realise this, algorithms must first be *parametrised* not only in p, but also in g, l, and available system memory. A thus-parametrised so-called *poly-algorithm* may then dispatch, at run-time, to the best-possible algorithmic variant– whether that is 2D, 3D, or something in-between. In the case of matrix multiplication, an immortal such poly-algorithm, capable of optimally traversing the memory-communication trade-off, is indeed realisable [MT99].

Generalisability. Analysing specific problems and classes of problems through their DAG structure is a well-known, tried and tested approach, applicable to a wide range of problems. Common DAG structures not discussed here include diamond DAGs for e.g. matrix decompositions [Tis02, Pet16, SLV18, BDG+18] and k-ary trees for e.g. broadcasts [PY88, GV94]. The analysis on cube DAGs can furthermore be extended to d-dimensional hyper-cubes for general tensor contractions and (thus) modern AI workloads. Algorithm analysis based on specific DAG structures avoids NP-hardness and inapproximability, leading to practical state-of-the-art algorithms and software without solving hard scheduling problems.

However, applying this approach requires a human in the loop, and come at the cost of problem-specificity. It requires significant expertise, compounding the human cost. Consider, for example, multi-dimensional FFTs, for which the de-facto standard today applies 1D FFTs in different directions, requiring data transpositions between directions when executing in parallel [PSFL20]. The full multi-dimensional FFT again corresponds to a butterfly DAGs, and by repeating the argument sketched earlier, a lower bound of n/p data movement in one superstep again emerges– even for higher-dimensional FFTs. However, a practical algorithm that matches these bounds was only recently discovered [KB23] –more than three decades after lower bound results that motivated BSP [Val90]– and required concepts from category theory to formulate [KBS25].

4.2. **Vertical data movement.** To recall, vertical data movement, or I/O, concerns data movement from slow to fast memory and assumes computations can only take place on data in fast memory. I/O is government by the red-blue pebbling game [HK81] in much the same way as horizontal data movement is governed by BSP. Here too, we may determine lower bounds and find algorithms that achieve them. This operates similarly as for horizontal computation in that specific DAG structures are typically analysed, resulting in lower bounds for, e.g., the fast Fourier transform (FFT), sorting, permuation, and transposition, may be found. Algorithms that match such bounds have been realised [AV88].

One common tool for proving lower bounds was introduced with the red-blue pebble game, and revolves around so-called *S-partitions*. Suppose the available fast memory is limited to r words. Then, considering a specific DAG structure, the strategy intuitively revolves around identifying sub-structures of the DAG that can be computed within r memory, yet are as large as possible. Finding the fewest such sub-structures that together cover all nodes in the DAG[5] recovers a partition that 1) completes the computation (due to the sub-structures covering all nodes), and 2) leads to a lower bound on I/O (due to finding the minimal required number of sub-structures). The earliest I/O bounds on matrix–matrix multplication, for example, follow this strategy [HK81, Tol99, ITT04], and these bounds are realisable by practical algorithms [RR52, MCJ69].

Generalisability. Lower bounds for I/O have been found for a variety of algorithms– canonical ones discovered in the past century [HK81, AV88, Tol99, ITT04], but also for recent algorithms such as the transformative FlashAttention [SY24]. Similar to established approaches for minimising communication, these methods are tried and tested, and lead to optimal sequential algorithms.

[5]while not introducing cycles

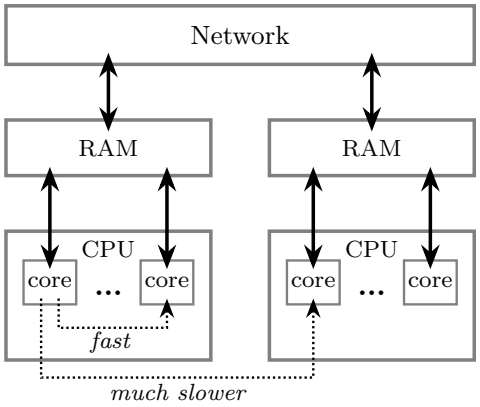


FIGURE 4.3. A non-uniform memory access architecture, corresponding to, for example, two single-socket systems connected through a network.

The classic red-blue pebble game does not account for partial computations where outputs may be computed while not all inputs are in fast memory simultaneously– see Section 3.4. The difference can be noticeable in practice: for matrix–vector multiplication with a vector of size n, for instance, accounting for partial computations can reduce the I/O cost by $n-1$ moves [PSY25]. We hence extend the S-partition notion to prove lower bounds in this partial pebbling game, and prove that the resulting tool retains similar utility as the original Hong and Kung one. Using it, indeed, we find lower bounds that take partial computations into account for the FFT, matrix–matrix multiplication, and FlashAttention [PSY25].

Although this approach hence too is generalisable across computations and refined models, individual results are specific to problems and require in-depth human expertise to establish. Jain and Zaharia suggest spectral partitioning methods to methodologically derive lower bounds on I/O costs for specific DAG structures [JZ20], potentially simplifying this effort– however, resulting bounds are not strict, meaning it may well reveal lower bounds that are not attainable in practice. More crucially, however, lower bounds on I/O are no longer sufficient in modern usage: contemporary systems are parallel and must be driven by parallel algorithms, and therefore horizontal communication must be considered also.

4.3. **NUMA.** Modern compute systems have increasingly deep memory hierarchies such as depicted by Figure 4.3. Moving data from a processor (i.e., one of the leafs) to a direct neighbour within the same subtree can be done at lower latency and higher throughput, when compared to moving the same data to a processor that is not in the same subtree. Contemporary CPUs, accelerators, and even system interconnects exhibit such non-uniform memory access (NUMA) behaviour, while the depth and width of the hierarchies tend to increase. We introduced models that combine horizontal and vertical data movement: MPP and Multi-BSP. Section 3.2 introduced these and established their hardness and inapproximability for memory hierarchies of some given depth d. We recall that Multi-BSP with depth $d = 1$, unit g, and zero latency reduces to MPP.

We may again consider specific DAG structures, prove lower bounds, and design algorithms and software that achieve them. Given the increased model complexity principally due their increased number of parameters from just one (r) and three (p, g, l) for pebbling and BSP, respectively, to approximately d and $4d$ parameters for MPP and Multi-BSP, this process becomes significantly harder in these more sophisticated models– and must furthermore be repeated for every workload of interest. Combating this complexity, Valiant proposes to relax the notion of optimality to be within a factor d, and provides a mathematical tool that simplifies determining lower bounds for arbitrary hierarchies. This tool applies for any $w(S)$-*limited straight-line program*: programs for which any subset of operations that takes at most S inputs and produces at most S outputs contains no more than $w(S)$ work. Several classic algorithms fall under this definition, for some of which he demonstrates deriving useful lower bounds: matrix–matrix multiplication, fast Fourier transform, and sorting [Val11].

5. IMPLICATIONS ON SOFTWARE DESIGN

We next discuss implications on the software design beyond algorithm-specific techniques such as parametrisation and poly-algorithms. Careful software design allows the automatic application of similar techniques that successfully avoid hardness to various degrees. We first discuss recursive and hierarchical

approaches and their limitations. Parallel frameworks and run-time systems allow for a natural layer in which some of the insights may be transparently applied, and are discussed second. Last, we discuss a class of computations that prevent avoiding hardness to significant degrees: irregular computations.

5.1. **Recursive and hierarchical approaches.** One idea decomposes the problem along a NUMA hierarchy, dividing the problem into sub-problems that each deal with the simpler uniform problem only. Assume there are $p = \Pi_i p_i$ processors in total, and p_i processors or sub-machines at each of the d levels of the NUMA hierarchy. Several possibilities for recursion or exploiting hierarchies exist, broadly classified as follows:

(1) treat each level individually, starting at the top (p_0), and applying recursion to each so-identified sub-problem ($p_{>0}$);
(2) schedule across p processors whilst ignoring hierarchies, then assign the p sub-schedules in a hierarchy-aware fashion;
(3) coarsen the problem DAG for the top-level problem, then progressively uncoarsen while iteratively considering the lower-level problems.

The difference between the first and third approach is that the former does not consider the complete problem on the lower-level problems (i.e., employing recursion), while the latter does consider the full computational DAG while refining the solution across a NUMA hierarchy (i.e., multi-level).

It is also possible to decompose different cost factors from the (hierarchical) models, so to then formulate approaches that perform orthogonal optimisation of the separated cost factors. For example,

(4) first minimise horizontal communication (minimise processor-local work, l, and g), and then given each processor-local DAG minimise vertical data movement;
(5) split the problem DAG into serial phases (minimise l), then parallelise each phase separately, and finally glue the sub-schedules together;
(6) find a good separation of the problem DAG in terms of work (e.g., minimise process-local work), and then find an optimised (hierarchical) communication schedule.

The fourth approach has both its steps take hierarchical costs into account– however, it separates different hierarchical factors: the first step accounts for shared resources (networks or shared caches) while the second step deals with private processor-local memory hierarchies. The fifth and sixth approaches only have one of its solution steps take a hierarchical system model into account.

A myriad of algorithms may be envisioned within each category. Section 6 details heuristic approaches of practical value in categories 3–6, while also presenting approaches for determining optimal schedules– including for hierarchical models. This section first follows with general results that warn against applying the other types of approaches in practice, and summarises their model-based evaluations.

1) Recursion. Consider first partitioning as a sub-class of scheduling problems, and consider a Multi-BSP (NUMA) hierarchy of depth $d = 2$. AJY: [be careful here given the unified definition of d (which is different from this one used here)– TODO] We showed that even when assuming optimal oracles for the uniform partitioning problem ($d = 1$), AJY: [same here] combining the optimal solutions in a recursive approach for $d > 1$ results in solutions that are up to $\Omega(n)$ away from optimal[6], where n is the number of vertices in the computational DAG [PAY23, Lemma 7.2]– i.e., the resulting distribution of the computation across the NUMA hierarchy can be arbitrarily bad. This does not mean recursion is an inappropriate tool; in fact, applying recursion to problem-specific DAGs may yield immortal algorithms [MT99]– recall also Section 4. However, for general-purpose software that exhibit computations with arbitrary structure, deciding how to distribute the computations over a NUMA hierarchy should never involve recursion.

2) Hierarchical assignment. Consider instead optimal uniform partitioning ($d = 1$), AJY: [definition of d has updated – TODO] followed by an optimal assignment of the parts through an arbitrary NUMA hierarchy ($d > 1$). If both steps can indeed be done optimally, the result is a g_0-approximation algorithm: the resulting distribution is off-optimal by a multiplicative factor proportional to the top-level data movement cost. Worst-case examples that achieve this bound can be constructed [PAY23, Theorem 7.4]. In a worst-case analysis, hierarchical assignment hence is preferable over recursion. However, the cost of ignoring hierarchy initially remains significant to the point that exploiting data locality lower in the hierarchy may be missed completely.

[6]the big-Omega notation means *at least* proportional to, in this case, the number of vertices in the DAG

3) Multi-level scheduling. These solutions take into account the full hierarchical description of the system, potentially during all of its phases: coarsening, solution refinement, uncoarsening, and the initial coarse-grained solve. Since during the whole process the entire hierarchy could be taken into account, no fundamental limits apply to this class of heuristic solution. Effectiveness, however, depends on specific algorithmic choices; the below summary includes model-based evaluations for specific such choices, and characterises solution quality over increasing d. Section 6 includes evaluation on real-world workloads.

4) Separating vertical from horizontal data movement. For general multi-level scheduling with $d = 2$ (i.e. MPP), AJY: [update to new definition of d– TODO] one of our more recent results shows that considering horizontal data movement separate from vertical data movement can lead to arbitrarily bad solutions: the gap to optimality may be a multiplicative factor proportional to at least n (i.e., $\Omega(n)$) [PBY25, Theorem 4.1]. Contemporary software stacks are nevertheless based on such a separation: the ScaLAPACK, for example, is a distributed-memory library for linear algebra solvers that relies on standard Basic Linear Algebra Subprograms (BLAS) [DDCHD90, CDPW92]. The latter are provided by vendors as highly-optimised sequential libraries, and include the optimisation of vertical data movement through a memory hierarchy– whereas the ScaLAPACK manages horizontal data movement. Our model-based evaluation summarised below shows that this separation may lead to performance loss, especially for larger d.

5) Communication scheduling. Consider starting from optimised work-to-processor assignments. Such assignments typically leave freedom of how to schedule inter-process data movement– required transfers may be overlapped with intermediate computation. While determining the optimal communication schedule is NP-hard [PAY25] –recall Section 3– this sub-problem is a useful refinement step in both single-shot as well as multi-level heuristics. The below summary as well as Section 6 make use of this.

6) Superstep scheduling. Assume that a given computation is already split into separate supersteps. To determine a parallel schedule, each superstep must be distributed across the p processors in a way that accounts for the optimised transfer of data between supersteps. This leads to a variant of the ϵ-balanced hypergraph partitioning problem, the *layer-wise balanced partitioning problem*. While again an NP-hard and inapproximable problem [PAY23, Theorem 5.2] we found a divide-and-conquer approach based on this separation effective for solving larger MPP problems [PBY25], with results as summarised below.

Evaluation. We employ model-based evaluation: given a computation and a schedule, as well as a computer system model (i.e., specific d and $(p_i, g_i, l_i)_k$ with $k \in \{0, 1, \ldots, d-1\}$), compute an abstract cost as prescribed by the Multi-BSP model. This allows evaluating heuristics on a set of benchmarks, and, for the smaller datasets, comparing heuristic solutions against optimal ones. Section 6 describes our heuristic approaches that combine the aforementioned categories 3–6, as well as describes our approaches for determining optimal schedules. Employing a database of computational DAGs from a wide range of applications [PAK+22, PAKY24], we first summarise observations of practical import to software design.

Evaluations on machine architectures of increasing NUMA depth $d \in \{1, 2, 4\}$ show improvements between 1.5–2.5× when evaluated[7] against traditional work-stealing approaches (the de-facto standard) as well as against state-of-the-art approaches [BJK+95, ZCL+22]. Crucially, multi-level methods work particularly well for $d = 4$, reaching speedups of 7.6× and 4.7× compared to 3.4× and 2.3× for our direct heuristics. For smaller d, however, the direct heuristics outperform the multi-level approach. The above results employ NUMA-aware schedulers– limiting ourselves to producing uniform schedules, speedups are 1.3–1.7× across all NUMA configurations and 1.7× and 1.4× for $d = 4$. The latter stands in stark contrast to the NUMA-aware schedules, demonstrating accounting for hierarchical data movement is crucial, especially with increasing system depth [PAKY24].

Our newest model-based evaluations concern schedules that optimise combined horizontal and vertical data movement for $d = 2$ (i.e., MPP). AJY: [update to newer definition of d–TODO] Evaluations show that decoupling minimising horizontal movement from vertical movement yields a 12% cost degradation, even when using an optimal BSP schedule and an optimal clairvoyant algorithm. This degradation increases to 34% when comparing against an online baseline: randomised work stealing and a standard LRU cache eviction policy. One caveat is that these results only involve the smallest DAGs from our benchmarks: computing optimal MPP schedules, even for modern commercial ILP solvers, are limited to $n = 80$. For larger computations, a divide-and-conquer approach allows deriving optimised MPP schedules for n up to 464, resulting in speedups between 0.88–1.5× versus the online baseline [PBY25]. Optimising vertical and horizontal data movement separately from one another hence can come at a significant performance penalty, and effective software approaches that co-optimise both should be investigated.

[7] we report here geometric means since we are comparing different workloads across different system configurations

AJY: [double-check that section 5.1 indeed contains what is below here] While for BSP finding optimal schedules for reasonably-sized DAGs is possible (see Section 6), doing the same for MPP is restricted to DAGs with tens of nodes only– and require in the order of several hours of compute time. This is unlikely to lead to satisfactory practical results, especially for $d > 2$ or when incorporating latency costs, even when allotting days of scheduling compute time. Recursive and hierarchical approaches may reduce the computational complexities of dealing with $d > 1$– Section 5.1 details those. AJY: [end double-check (but note that a remainder part should be double-checked to be contained in Section 6]

5.2. **Run-time systems.** Given a NUMA system such as depicted by Figure 4.3, one intuitive idea realises a matching hierarchical run-time system (RTS). Realising this in pure software results in different groups of processors, each controlled by a separate run-time system instance. Hardware schedulers strategically placed within the system hierarchy could lessen scheduling overheads. No matter the realisation, however, hierarchical scheduling should bear in mind the above results.

A hierarchical run-time system design where the top-level dispatches tasks to lower-level RTSes, for example, can result in arbitrarily bad schedules [PAY23, PBY25]. Working around this limitation requires holistic scheduling approaches that consider the global state of the computation, and requires passing the lower-level RTSes information on what parts of the computation their peer holds. This need not be a full copy of how the global computation is distributed; instead, sketches of what other RTSes contain could be maintained. This results in a distributed run-time system where different components are aware of others' responsibilities, mimicking human organisation, for which we coin the phrase *federated run-times*.

Consider, as a concrete example, the techniques that exploit specific computational DAG structures outlined throughout Section 4. Employing recursion to hierarchically distribute a cube DAG across a matching NUMA hierarchy, for example, leads to an immortal algorithm [MT99, Val11]. Mapping this to federated run-times requires different RTSes at different levels in the hierarchy have the following information: that the global computation matches a cube DAG of some size n, full information about the sub-hierarchy of their parent (d and the various p_i, g_i, l_i), and its own position in the hierarchy.

Practical realisations of this idea in RTSes exist e.g. for linear pipeline DAGs [MG18]. Linear pipelines occur naturally when dispatching computations to remote resources, in serving large-scale language models, with on-line streaming services in general, and in linear algebra computations sparse and dense [MAY23, SJZY23]. This requires parametrisation not only within algorithms (allowing algorithms to adapt to systems; i.e., poly-algorithms), but also that algorithms drive federated run-time systems through parametrisation. This warrants a systematic separation of functional semantics from parametrisation, as argued by Mastoras and Yzelman for linear pipelines through annotations [MY23]. Applying this approach to general-purpose programming on federated systems, however, requires similar investigations for other DAG structures and could enjoy more systematic study by research and industry.

5.3. **Programming frameworks.** Lower-level established programming models often employed include POSIX Threads, OpenMP, and MPI [IEE24, Ope24, For25], and allow expert programmers to achieve highest performance and optimal scalability. Modern programming frameworks instead tailor themselves to the humble programmer [Dij72], emphasising end-user productivity: resulting in higher-level frameworks such as Pregel [MAB+10, CEK+15], Scikit-learn [PVG+11], Spark [ZXW+16], TensorFlow [ABC+16], and PyTorch [PGM+19]. These frameworks often still achieve state-of-the-art performance, on-par with codes hand-written by the hero programmer in scalability and performance [Yze24].

Novel accelerators and custom interconnects inspire departures from established programming frameworks, leading to novel frameworks both hero and humble [SY19, TKC19, YDNNS20, Yze22, WCS+25, NVI25, Hua25]. The regular surfacing of novel programming frameworks translates to an opportunity: sufficiently high-level interfaces allow for the automatic application of the techniques this paper summarises. Frameworks like Cyclops and Algebraic Programming (ALP), for example, employ the optimality results on horizontal communication for cube DAGs from Section 4 for both dense linear algebra computations and tensor computations [SMHD13, SJZY23]. ALP furthermore breaks with the classic separation between coarse-grained BLAS calls and MPI calls [CDPW92] by splitting matrices in smaller tiles that fit private caches. While BLAS use remains for the small tiles, the framework controls both horizontal and vertical data movement [MAY23, SJZY23]. Similar techniques have shown to benefit run-time system integration [DFH+13]. ALP also pioneers the automatic selection of performance parameters to the underlying run-time system [MAY23, AJY26], thus transparently applying the techniques from Section 5. Early research suggests similar techniques apply to AI frameworks [GKA+21].

Frameworks may also find cause to handle general scheduling problems, optimising schedules either as part of a compilation step or at run time. Two important scenarios could warrant such a decision.

First, for the so-called *irregular* class of computations, manual efforts are often outperformed by heuristic approaches to finding optimised schedules. This succeeds due to finding latent structures in underlying computation graphs, structures that are not known a priori. This approach is common within sparse computations [BFAY+12]. Second, an application might repeat the same computation many times, yet escape manual analysis due to the complexities involved or due to the underlying computational graph varying in practice. Examples include inference serving, training large-scale AI models, and investigating alternative neural network designs or AI operators. Rather than awaiting repeated significant human investment to find lower bounds, optimal algorithms, appropriate parametrisations, and integration with production software, it may instead be appealing to automate this search and repeatedly solve the underlying scheduling problems. PyPTO is an example of such a framework [Hua25], and Section 6 overviews both optimal approaches for small-scale problems as well as ones for larger-scale ones.

AJY: [TODO: double-check that the below is already in this Section somewhere] Even so, both theoretical and experimental study of these models still offer valuable insights. For instance, one can analyze the difference between holistic scheduling strategy that aims to find a solution for the entire problem, and a two-stage approach that first comes up with a good parallelization strategy, and then combines this with a cache management strategy to optimize I/O costs. On the theoretical side, one can show a worst-case example where the two-stage schedule is a very large factor away from the actual optimum, even if both stages are optimal separately. On the experimental side, one can confirm that the holistic view can indeed outperform the two-stage approach by a small margin [PBY25]. AJY: [double-check that what is in this paragraph is already conveyed by Section 5, and if not, move it there]

6. OneStopParallel

Describe what OSP contains and aims to provide/support

6.1. Optimal Scheduling – ILPs.

Given a scheduling problem, the most desirable outcome would be to find the optimal schedule (with an actual guarantee that it is optimal). As we have discussed, this is not possible in general for NP-hard problems. However, this only means that there is no algorithm that efficiently finds the optimum *for every instance*. In practice, even for NP-hard problems, the community has developed very sophisticated tools that still find the optimum relatively fast in many cases, especially in problems derived from practice, which tend to be much more structured than random instances.

One particularly well-studied NP-hard problem is Integer Linear Programming (ILP), which consists of

- a set of variables, e.g. x_1, x_2, x_3, some of which might take any real value, whereas others might be restricted to the set of integers, or to the boolean set $\{0, 1\}$;
- a set of linear inequality (or equality) constraints on these variables, e.g. $x_1 \geq 0$, or $x_1 + 3x_2 - 5x_3 \geq -7$;
- a linear objective function on these variables, e.g. $2x_1 - x_2 + 4x_3$, which we want to either minimize or maximize.

If some of the variables are restricted to integers or boolean values, then this problem is NP-hard. What makes ILPs particularly interesting is that due to their general formulation, many combinatorial problems, including scheduling, can be expressed as ILPs. Furthermore, there are numerous highly optimized (open-source and commercial) ILP solvers; for instance, the Cardinal Optimizer (COpt) commercial solver used in OneStopParallel [GHW+23]. If we find a suitable ILP representation for our scheduling problem, then these ILP solvers can be directly utilized to find optimal schedules.

BSP scheduling indeed has a natural, if somewhat complicated, representation as an ILP. This is organized around binary variables $\text{COMPUTED}_{v,p,s}$ for each vertex v, processor p and superstep s, indicating whether vertex v is computed on processor p in superstep s. With these variables, we can, for example, create for each vertex v a linear constraint $\sum_p \sum_s \text{COMPUTED}_{v,s,p} = 1$ to express that the vertex must be computed on exactly one processor in exactly one superstep. Other central variables are $\text{PRESENT}_{v,p,s}$ indicating whether the output of v is present on p by superstep s, and $\text{SENT}_{v,p_1,p_2,s}$ indicating whether the output of v is sent from processor p_1 to processor p_2 in superstep s. Using these variables, one can encode all the validity conditions of a BSP schedule as linear constraints. The ingredients of the BSP cost function can also be expressed with auxiliary variables and constraints, and hence the objective of the resulting ILP is equivalent to minimizing the cost in the corresponding scheduling problem [PAY25, PAKY24].

Using the ILP solver on this formulation allows to find the optimal schedule for smaller graphs, sometimes up to several hundred nodes. Even on slightly larger graphs, the solver is often able to the

find the optimal or close-to-optimal solutions if we can afford to run it for several hours. As such, this ILP-based approach is a very convenient way to find optimal schedules when we have important workloads of smaller size.

Besides finding the optimal solution, one of the main advantages of this approach is adaptability. Even when focusing on BSP-like models, one encounters countless slightly different versions of the scheduling problem in different application domains. For instance, some scheduling problems might have core type restrictions for an architecture with heterogeneous cores; or allow recomputation in order to decrease costs,; or allow to overlap computations and communications. If we have a specialized scheduling algorithm, it often takes significant effort to adapt it from one of these problem variants to another. In contrast, with ILPs, this can usually be solved with minor modifications to the variables and constraints. As such, using ILPs is a very convenient approach to develop scheduling algorithms that can be quickly tailored to a new application domain if desired.

6.2. **Heuristics.** The ILP approach excels at finding optimal or close-to-optimal solutions for smaller problem instances. However, in many domains, the input DAGs are significantly larger, up to millions of nodes, and we are required to find good schedules for these in seconds or fractions of a second. In this case, we instead need to turn towards sophisticated scheduling heuristics.

OSP incorporates a wide range of large-scale heuristics that can find good schedules in DAGs up to hundreds of millions of nodes very quickly [PAKY24, BPSY25]. Many of these are based on so-called list scheduling, where the assignments are based on priorities are assigned to each node. In particular, OSP introduces the concept of *barrier list schedulers*, adjusting the list scheduling approach to the BSP model with synchronization barriers, where there is an inherent trade-off between maintaining enough parallelism in the schedule but also minimizing the number of barriers used. OSP implements several different algorithms following this barrier list scheduling framework. These algorithms variants are tailored towards different applications where different problem aspects are dominant: e.g., they specifically prioritize maintaining a better work balance, or reducing the total amount of communicated data, or minimizing the number of synchronization barriers, or maintaining good cache locality during the computation.

Given an initial schedule, another important class of heuristics is so-called local search algorithms that execute small local improvements on this solution to further reduce its cost. A local search algorithm can, for instance, take an arbitrary node of the DAG, and try to move this to a different processor or different superstep, while leaving the rest of the assignment unchanged 6.1. A straightforward local search algorithm (also called hill climbing) considers such small modification, and if it leads to a (still valid) solution with lower cost, then executes the change, and continues with this modified schedule. The algorithm goes on until either none of the possible modification steps improve the current solution (a so-called local minimum is reached), or until a predefined time limit is reached.

Besides hill climbing, OSP also implements a more sophisticated local search algorithm (named Kernighan-Lin method) that is also able to escape local minima to some extent. In particular, the Kernighan-Lin scheduler occasionally also allows to execute moves that increase the cost, or make the schedule slightly invalid, in the hope of finding a better (and again valid) solution after multiple steps. Techniques like this are crucial to ensure that local search algorithms can also improve on the solution when this would be possible with a larger modification to the schedule, but not through a series of smaller changes.

6.3. **Combined algorithms.** The best method to obtain a high-quality schedule is often to combine several of the algorithms above. In the simplest case, this can simply be a sequential combination: we first obtain an initial solution from a barrier list scheduler, we improve this with a local search algorithm, and then feed this as an initial solution to the ILP solver [PAKY24]. However, there are also more sophisticated ways to use the algorithms above as ingredients in a more complex scheduler.

Besides the algorithms, another vital ingredient in these combined methods are DAG coarseners, which contract smaller clusters of nodes into a single node, while ensuring that the resulting DAG still remains acyclic. The goal of these algorithms is to significantly decrease the size of the DAG, while ideally retaining most of its original structure. In particular, coarseners mostly aim to identify denser cluster of nodes in the DAG that are connected by many or heavy-weighted edges. Merging these clusters into a single node inherently decreases the amount of communication required between processors, and naturally improves data locality.

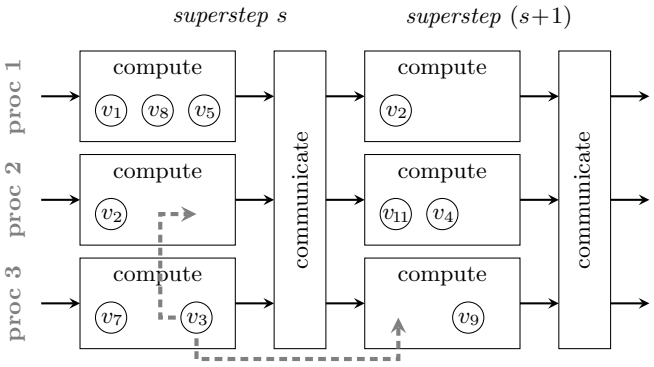


FIGURE 6.1. Illustration of two possible improvement steps in a local search algorithm: moving the execution of node v_3 to processor 2 in the same superstep, or keeping it on processor 3, but moving it to (the computation phase of) the next superstep. In hill climbing, a move is only executed if it produces a new schedule that is still valid, and has lower cost than the current schedule.

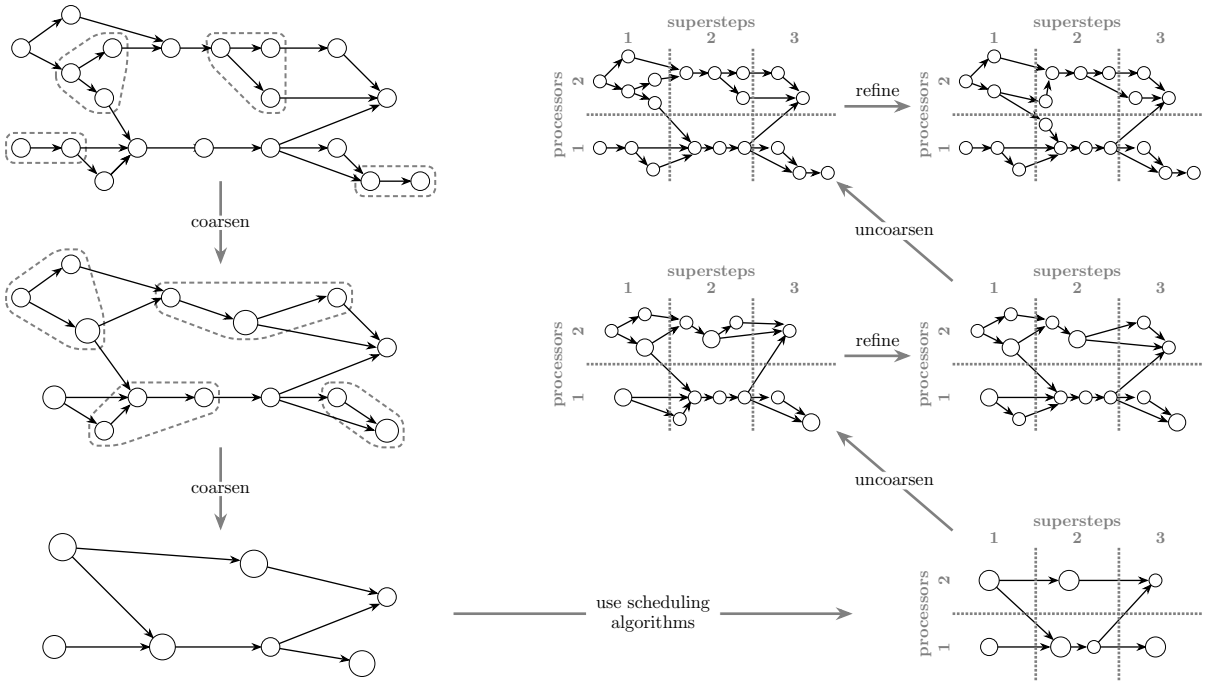


FIGURE 6.2. Illustration of multilevel scheduling: the DAG is first coarsened iteratively to obtain a much smaller DAG. A scheduling algorithm is then applied to this coarse DAG. Finally, the DAG is iteratively uncoarsened to the original DAG. After each uncoarsening step, the schedule is further refined with an iterative improvement (e.g. local search) algorithm.

OSP implements several acyclic coarsening algorithms that can be used as a pre-processing step before applying the schedulers. This idea can also be extended into the so-called multilevel, or coarsen-solve-refine paradigm, which is widely used in hypergraph partitioning [KAKS99, PSSS21]. The key idea here is to begin with a coarsening phase that acyclically coarsens the DAG in several step iteratively. Then in the solving phase, we apply our previous scheduling algorithms to find a good schedule for this (much simpler) coarsened DAG. Finally, in the refinement phase, we extend this schedule to the original, uncoarsened DAG in smaller steps, through a series of DAGs of increasing size. After each uncoarsening step, an iterative improvement algorithm (like local search) is also applied in order to further refine the schedule to better fit the newly uncoarsened part of the DAG. The approach is illustrated in Figure 6.2.

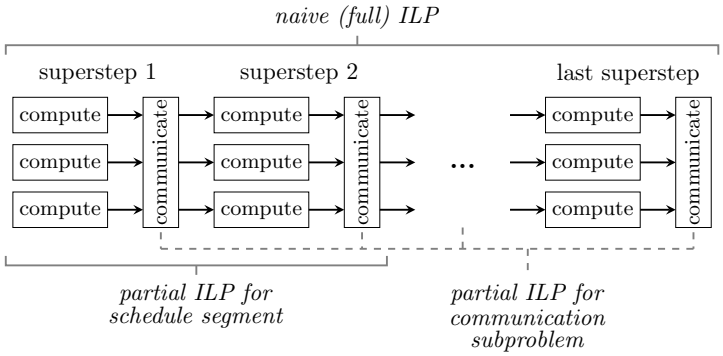


FIGURE 6.3. Illustration of ILP-based algorithms for BSP scheduling. The methods use an ILP representation to capture (i) the entire scheduling problem, (ii) a smaller segment of supersteps in the schedule, or (iii) the subproblem of optimizing communication steps.

Similarly, there are also several ways to apply the ILP-based scheduling idea to larger DAGs. For instance, given an already found BSP schedule, we can split it into segments of supersteps (e.g. supersteps 1–4, supersteps 5–8, etc.), and represent these as smaller ILP problems, in order to reoptimize and improve specific parts of the schedule. Similarly, the subproblem of scheduling communication steps can be captured as a notably easier ILP problem, which can again allow us to somewhat decrease the total cost (see Figure 6.3). While these methods do not guarantee an optimal solution like the full ILP, they provide significant opportunities to improve the schedules for DAGs up to thousands of nodes [PAKY24].

In an even more advanced approach, we can apply divide-and-conquer algorithms that try to split that problem into several, relatively independent sub-problems. For instance, one can apply an acyclic partitioning algorithm to acyclically split the input DAG into several smaller parts such that the number of edges connecting different parts is as small as possible. A scheduling algorithm (possibly with ILPs) can then find a separate schedule for each of the parts independently, or in a topological order such that it also takes into consideration the sub-schedules developed for previous parts. The sub-schedules are then merged into a single solution in the end, possibly with some post-processing steps or local search algorithms to further improve the combined schedule [PBY25].

6.4. **Other features.** Besides having a wide range of schedulers, OSP also ensures that these schedulers can be used in different variants of the scheduling problem. For instance, most algorithms can directly optimize for communication costs with NUMA effects, or consider processor type restrictions in a heterogeneous architecture, where vector and tensor operations must be assigned to vector and tensor cores, respectively. There are also complex heuristics to introduce replication steps into a schedule to further decrease communication costs, or to try to overlap computation and communication steps in an existing schedule.

Apart from the algorithms themselves, OSP also has a range of further features to support the study of scheduling problems. It incorporates a test suite to evaluate the algorithms, tools that allow to visualize the input DAGs or the developed schedules, and file readers/writers in several different formats to load the the input DAGs and save the output schedules.

6.5. **Access through a web interface.** Besides the command-line and programmatic interfaces, we also provide a lightweight web interface to OSP. The goal of this interface is not to replace the full toolbox, but to make the most common scheduling workflows directly accessible to users who want to experiment with OSP on their own DAGs without first integrating the library into a larger software stack. In particular, the interface allows users to upload a computational DAG together with a machine description, select one or more schedulers, run the corresponding OSP executable on the server, and compare the resulting schedules through key metrics such as total cost, work cost, communication cost, number of supersteps, and scheduling time.

The web interface furthermore supports downloading the generated schedules, and can thus serve both as an entry point for new users and as a rapid experimentation environment when comparing scheduler behaviour on representative workloads. This is particularly useful in early-stage application studies, where one often wants to evaluate several scheduling strategies quickly before deciding whether a tighter integration of OSP into an application or runtime system is warranted.

TODO add an image of a visualized schedule here?

TODO mention partitioning, pebbling algorithms? Spectral stuff? Ubuntu /Spack package?

PAP: [Determining optimal schedules using ILP formulations for partial pebbling is at least as hard as the non-partial case (?) - needed?] AJY: [my take on this is it depends on what you mean – if it's a hardness result, it should go to Section 3. If it's a practical observation on ILP formulations, it should go to Section 6 here?]

7. Results

Summarise key results from the various publications and applications

- EDA / SpTrsv
- MoE
- algorithm-specific parallelisation should be pointed out? (FFT, GEMM, Attention?, etc.)

7.1. **Sparse Triangular Solve.** In a wide range of applications from structural and electromagnetic simulations, found in engineering to electronic design automation, to Artificial Intelligence, one is often faced with the problem of finding $\boldsymbol{x}$ satisfying a linear equation

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}\,. \tag{7.1}$$

The golden rule when faced with such a problem is:

"Don't invert that matrix." — *John D. Cook.*

Instead, one employs what is called a *solve* operation which only computes the vector $\boldsymbol{x}$ which is only a fraction of the information contained in the inverse $\boldsymbol{A}^{-1}$. Even with this optimisation, such a solve is often the bottleneck of a simulation. There are several reasons for this

(1) numerical stability,
(2) many fine dependencies, and
(3) short operations.

To illustrate, we consider the special case where $\boldsymbol{A}$ is lower triangular we furthermore assume that $\boldsymbol{A}$ is sparse. Albeit it being a special case, very often the general case can be reduced to this case. Here is an example of such a matrix

$$\boldsymbol{A} = \begin{pmatrix} A_{1,1} & & & & \\ & A_{2,2} & & & \\ A_{3,1} & & A_{3,3} & & \\ & A_{4,2} & & A_{4,4} & \\ & A_{5,2} & A_{5,3} & & A_{5,5} \end{pmatrix} = \begin{pmatrix} 1.34 & & & & \\ & 0.38 & & & \\ -0.86 & & 7.01 & & \\ & -0.26 & & -0.63 & \\ & 3.75 & -7.33 & & 1.21 \end{pmatrix}, \tag{7.2}$$

where we omitted the zero entries. The standard algorithm to solve the linear equation, Eq. (7.1), is the forward-substitution algorithm, see Algorithm 7.1.

Algorithm 7.1: Forward-substitution algorithm

Data: An invertible lower-triangular matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ and a vector $\boldsymbol{b} \in \mathbb{R}^n$.
Result: A vector $\boldsymbol{x} \in \mathbb{R}^n$ such that $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

```
1 for i = 1, ..., n do
2     t ← b_i
3     for j = 1, ..., i − 1 with A_{i,j} ≠ 0 do
4         t ← t − A_{i,j} x_j
5     x_i ← t / A_{i,i}
```

We see that each non-zero entry $A_{i,j}$ of $\boldsymbol{A}$ corresponds to an operation in the algorithm. Specifically, if $i < j$ the entry $A_{i,j}$ corresponds to Line 4 and if $i = j$ it corresponds to Line 5. Implicitly, there are a lot of dependencies in the algorithm. To execute Line 4 with $A_{i,j}$ we need to know the value of x_j which corresponds to the entry $A_{j,j}$. Likewise, to produce the value x_i corresponding to the entry $A_{i,i}$, we need to have done the accumulation in Line 4. Hence, it depends on the computations corresponding to the entries $A_{i,j} \neq 0$ with $j < i$. For the matrix $\boldsymbol{A}$ in Equation 7.2, these dependencies are illustrated in Figure 7.1.

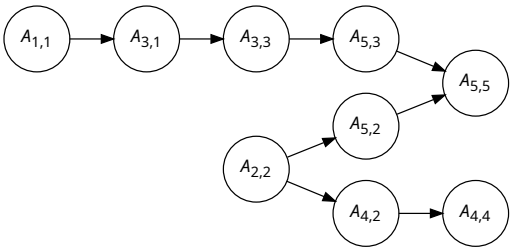


FIGURE 7.1. Dependencies in the forward-substitution algorithm, Algorithm 7.1, of the matrix $\boldsymbol{A}$ in Equation 7.2.

If one wants to parallelise this algorithm, one has little choice to study the dependencies. At the Huawei Von Neumann Research Center in Zurich, we have developed algorithms to do precisely that. The algorithms *GrowLocal* and *GrowLocal SSP*, available in OneStopParallel [BLM+24], clusters operations in a manner that preserves locality, maximises independent compute all the whilst keeping significant parallelism. In Figure 7.2, we show the geometric mean speed-up including interquartile ranges of our scheduling algorithms *GrowLocal* and *GrowLocal SSP* and compare them against the state-of-the-art schedulers HDagg (synchronous scheduler) [ZCL+22] and SpMP (asynchronous scheduler) [PSSD14]. The benchmarks are over various CPU architecture, including a Huawei Kunpeng, and three data sets derived from the SuiteSparse Matrix Collection [DH11]. The first data set, Suite Sparse, represents a hard to parallelise data set and the other two represent typical matrices one would find in applications.

Given the success of our scheduler, we have applied it to the Huawei in-house EDA tooling. In Figure 7.3, we show the speed-ups we get on EDA matrices on a Huawei Kunpeng.

FIGURE 7.2. Scaling plots depicting the geometric mean speed-up over Serial and interquartile range of each algorithm on a given data set, architecture, and core count.

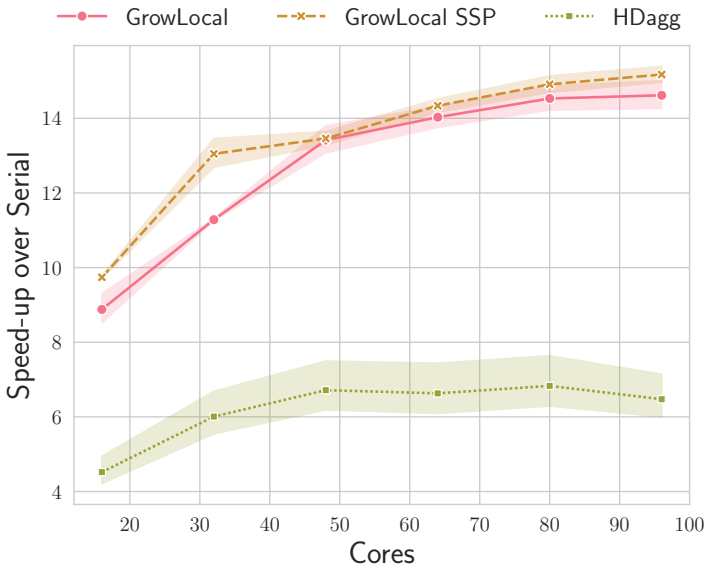


FIGURE 7.3. Scaling plots depicting the geometric mean speed-up over Serial and interquartile range of each algorithm on EDA matrices on an ARM Huawei Kunpeng machine.

7.2. **Mixture of experts.** In a Mixture-of-Experts (MoE) model, each layer contains many potential experts, but during a single inference pass only a small subset of these experts is activated for each input. This sparse activation pattern means that, at runtime, the model does not use all experts simultaneously. Instead, it dynamically selects only a few experts per layer based on the input. Since different prompts trigger different combinations of experts, the activation frequency of each expert group can vary widely.

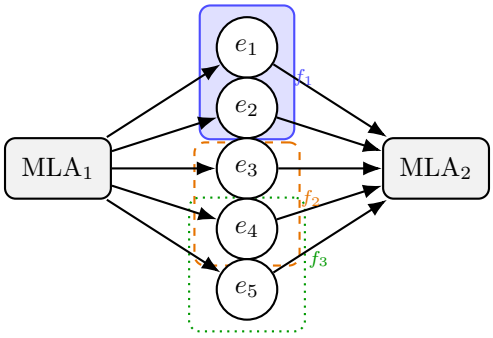


FIGURE 7.4. Simplified hypergraph of two MoE layers. Each colored group has its own frequency f_e. The ILP uses these frequencies to decide which experts and MLAs should be placed on the same device.

Profiling these activation patterns across a large dataset of user prompts allows us to identify which groups of experts are frequently selected together. This information is crucial for model distribution: experts that often co-activate can be co-located on the same device to reduce cross-device communication, while rarely co-activated experts can be placed independently to improve memory usage and computational load balancing.

To incorporate this profiled activation-frequency data into the placement problem, we transform the model's computational graph into a hypergraph. A hypergraph is a generalization of a graph in which each edge, called a hyperedge, can connect any number of nodes rather than exactly two. In our formulation, nodes represent experts in the model, and each hyperedge represents a set of experts activated together during a single inference step. The weight of a hyperedge, f_e, is the observed activation frequency of that group, estimated by profiling the model on a dataset of real user prompts. In this way, the hypergraph directly encodes how often different groups of experts are used in practice, providing the basis for efficient MLA and expert placement decisions.

Given this hypergraph representation, we formulate the placement problem as an ILP that jointly determines where experts and MLA blocks should be placed. The objective is to minimize the connectivity, or $(\lambda - 1)$, metric. For each hyperedge e, let λ_e denote the number of devices intersected by that hyperedge. If $\lambda_e = 1$, all experts activated together are placed on the same device and no cross-device communication is required. If $\lambda_e > 1$, the request may require data from multiple devices, producing a communication cost proportional to $\lambda_e - 1$. Therefore, minimizing this metric encourages frequently co-activated experts to be placed together, especially when their hyperedge has a large activation frequency f_e.

The ILP uses binary decision variables to assign model components, such as experts and MLA blocks, to devices. These placement variables also determine the communication pattern implicitly: whenever components that are used together are placed on different devices, data movement is required. Thus, the optimization does not need to model every data transfer explicitly; instead, communication is captured through the hypergraph connectivity induced by the placement.

The formulation is subject to several placement and resource constraints. By default, each expert is assigned to exactly one partition, although this restriction can be relaxed when expert replication is allowed. Each device must also respect its memory capacity M, which is especially important for MoE models because expert weights are large in aggregate and KV caches may vary dynamically with the workload.

Finally, the formulation accounts for MLA placement. Since MLA blocks should preferably be close to the experts they serve, this preference can be modeled either by including MLA blocks in the corresponding hyperedges or by adding a penalty term that discourages separating them from important expert groups. The strength of this penalty can be scaled by the activation frequency of the corresponding hyperedge. In addition, each MLA block is required to be replicated on at least `MIN_REPLICAS` and

at most `MAX_REPLICAS` devices, while an additional constraint prevents placing multiple replicas of the same MLA block on a single device.

8. Conclusion

In conclusion, ... be careful with multi-level (distributed) scheduling

For important kernels or classes of computation, nothing beats manual work (yet:)– lower bounds, trade-off identifications, (poly-)algorithms that achieve them, combined with engineering efforts. Getting bounds though should not be skipped (engineers need to know what to aim for)

Beyond its role as a research toolbox, OneStopParallel is also meant to be directly usable in practice: users can integrate the underlying schedulers into applications and runtimes, or use the web interface as a lightweight *OSP-as-a-Service* environment for testing and exploratory evaluation on concrete workloads.

8.1. **Future directions.**

Parametric hardness? There was some nice work at SLS26

Extensions to OSP – mention collab with Dimos and the database

References

[ABC+16] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: a system for large-scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.

[ACS90] Alok Aggarwal, Ashok K. Chandra, and Marc Snir. Communication complexity of PRAMs. *Theoretical Computer Science*, 71(1):3–28, 1990.

[AJY26] Petros Anastasiadis, D. Jelovina, and A. N. Yzelman. PaHiS: A hierarchical synchronous parallel model for irregular workloads. Submitted, 2026.

[AV88] Alok Aggarwal and S. Vitter, Jeffrey. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[BDG+18] Grey Ballard, James Demmel, Laura Grigori, Mathias Jacquelin, and Nicholas Knight. A 3D parallel algorithm for QR decomposition. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures*, pages 55–65, 2018.

[Ber89] Jarle Berntsen. Communication efficient matrix multiplication on hypercubes. *Parallel Computing*, 12(3):335–342, 1989.

[BFAY+12] Rob H Bisseling, BO Fagginger Auer, AN Yzelman, Tristan van Leeuwen, Umit V Catalyurek, et al. Two-dimensional approaches to sparse matrix partitioning. 2012.

[BJK+95] Robert D Blumofe, Christopher F Joerg, Bradley C Kuszmaul, Charles E Leiserson, Keith H Randall, and Yuli Zhou. Cilk: An efficient multithreaded runtime system. *ACM SigPlan Notices*, 30(8):207–216, 1995.

[BLM+24] Toni Böhnlein, Benjamin Lozes, Christos K. Matzoros, Pál András Papp, and Raphael S. Steiner. OneStopParallel. https://github.com/Algebraic-Programming/OneStopParallel, 2024.

[BPSY25] Toni Böhnlein, Pál András Papp, Raphael S. Steiner, and A. N. Yzelman. Efficient parallel scheduling for sparse triangular solvers. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1263–1265, 2025.

[BPY25] Toni Böhnlein, Pál András Papp, and A. N. Yzelman. Red-blue pebbling with multiple processors: Time, communication and memory trade-offs. In *International Colloquium on Structural Information and Communication Complexity (SIROCCO)*, pages 109–126. Springer, 2025.

[CDPW92] Jaeyoung Choi, Jack J Dongarra, Roldan Pozo, and David W Walker. ScaLAPACK: A scalable linear algebra library for distributed memory concurrent computers. In *[Proceedings 1992] The Fourth Symposium on the Frontiers of Massively Parallel Computation*, pages 120–127. IEEE, 1992.

[CEK+15] Avery Ching, Sergey Edunov, Maja Kabiljo, Dionysios Logothetis, and Sambavi Muthukrishnan. One trillion edges: Graph processing at Facebook-scale. *Proceedings of the VLDB Endowment*, 8(12):1804–1815, 2015.

[DDCHD90] Jack J Dongarra, Jeremy Du Croz, Sven Hammarling, and Iain S Duff. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)*, 16(1):1–17, 1990.

[DFH+13] Jack Dongarra, Mathieu Faverge, Thomas Herault, Mathias Jacquelin, Julien Langou, and Yves Robert. Hierarchical QR factorization algorithms for multi-core clusters. *Parallel Computing*, 39(4-5):212–232, 2013.

[DH11] Timothy A. Davis and Yifan Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software (TOMS)*, 38(1):1–25, 2011.

[Dij72] Edsger W Dijkstra. The humble programmer. *Communications of the ACM*, 15(10):859–866, 1972.

[DNS81] Eliezer Dekel, David Nassimi, and Sartaj Sahni. Parallel matrix and graph algorithms. *SIAM Journal on Computing*, 10(4):657–675, 1981.

[For25] MPI Forum. MPI: A message-passing interface standard version 5.0. Technical report, June 2025.

[GGK02] Ananth Y Grama, Anshul Gupta, and Vipin Kumar. Isoefficiency: Measuring the scalability of parallel algorithms and architectures. *IEEE Parallel & Distributed Technology: Systems & Applications*, 1(3):12–21, 2002.

[GHW+23] Dongdong Ge, Qi Huangfu, Zizhuo Wang, Jian Wu, and Yinyu Ye. Cardinal Optimizer (COPT) user guide. https://guide.coap.online/copt/en-doc, 2023.

[GKA+21] Evangelos Georganas, Dhiraj Kalamkar, Sasikanth Avancha, Menachem Adelman, Cristina Anderson, Alexander Breuer, Jeremy Bruestle, Narendra Chaudhary, Abhisek Kundu, Denise Kutnick, et al. Tensor processing

primitives: A programming abstraction for efficiency and portability in deep learning workloads. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, 2021.

[GV94] Alexandros V. Gerbessiotis and Leslie G. Valiant. Direct bulk-synchronous parallel algorithms. *Journal of parallel and distributed computing*, 22(2):251–267, 1994.

[HK81] Jia-Wei Hong and Hsiang-Tsung Kung. I/O complexity: The red-blue pebble game. In *Proceedings of the 13th ACM Symposium on Theory of Computing (STOC)*, pages 326–333, 1981.

[Hoc94] Roger W Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel computing*, 20(3):389–398, 1994.

[Hua25] Huawei Technologies Co., Ltd. Huawei PyPTO: Parallel tile operator framework, 2025.

[IB01] Márcia A. Inda and Rob H. Bisseling. A simple and efficient parallel FFT algorithm using the BSP model. *Parallel Computing*, 27(14):1847–1878, 2001.

[IEE24] IEEE Standards Association. IEEE Standard for Information Technology. Technical Report IEEE Std 1003.1-2024, IEEE, New York, NY, USA, 2024.

[ITT04] Dror Irony, Sivan Toledo, and Alexander Tiskin. Communication lower bounds for distributed-memory matrix multiplication. *Journal of Parallel and Distributed Computing*, 64(9):1017–1026, 2004.

[JZ20] Saachi Jain and Matei Zaharia. Spectral lower bounds on the I/O complexity of computation graphs. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures*, pages 329–338, 2020.

[KAKS99] George Karypis, Rajat Aggarwal, Vipin Kumar, and Shashi Shekhar. Multilevel hypergraph partitioning: Applications in VLSI domain. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 7:69–79, 1999.

[KB23] Thomas Koopman and Rob H. Bisseling. Minimizing communication in the multidimensional FFT. *SIAM Journal on Scientific Computing*, 45(6):C330–C347, 2023.

[KBS25] Thomas Koopman, Rob H. Bisseling, and Sven-Bodo Scholz. Category theory for supercomputing: The tensor product of linear BSP algorithms. *arXiv preprint arXiv:2510.00979*, 2025.

[MAB+10] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146, 2010.

[MAY23] Aristeidis Mastoras, Sotiris Anagnostidis, and A. N. Yzelman. Design and implementation for nonblocking execution in GraphBLAS: Tradeoffs and performance. *ACM Trans. Archit. Code Optim.*, 20(1), March 2023.

[MCJ69] A. C. McKellar and Edward G. Coffman Jr. Organizing matrices and matrix operations for paged memory systems. *Communications of the ACM*, 12(3):153–165, 1969.

[MG18] Aristeidis Mastoras and Thomas R Gross. Understanding parallelization tradeoffs for linear pipelines. In *Proceedings of the 9th International Workshop on Programming Models and Applications for Multicores and Manycores*, pages 1–10, 2018.

[MT99] W. F. McColl and A. Tiskin. Memory-efficient matrix multiplication in the BSP model. *Algorithmica*, 24:287–297, 1999.

[MY23] Aristeidis Mastoras and Albert-Jan N Yzelman. Studying the expressiveness and performance of parallelization abstractions for linear pipelines. In *Proceedings of the 14th International Workshop on Programming Models and Applications for Multicores and Manycores*, pages 29–38, 2023.

[NVI25] NVIDIA Corporation. cuTile: A tile-based programming model for NVIDIA GPUs. https://github.com/nvidia/cutile-python, 2025. Accessed: 2026-04-06.

[ÖBU+19] M Yusuf Özkaya, Anne Benoit, Bora Uçar, Julien Herrmann, and Ümit V Çatalyürek. A scalable clustering-based task scheduler for homogeneous processors using dag partitioning. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 155–165. IEEE, 2019.

[Ope24] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification, Version 6.0*, Nov 2024.

[PAK+22] Pál András Papp, Georg Anegg, Aikaterini Karanasiou, A. N. Yzelman, and Mirko De Vita. HyperDAG database. https://github.com/Algebraic-Programming/HyperDAG_DB, 2022.

[PAKY24] Pál András Papp, Georg Anegg, Aikaterini Karanasiou, and A. N. Yzelman. Efficient multi-processor scheduling in increasingly realistic models. In *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 463–474, 2024.

[PAY23] Pál András Papp, Georg Anegg, and A. N. Yzelman. Partitioning hypergraphs is hard: Models, inapproximability, and applications. In *35th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 415–425, 2023.

[PAY25] Pál András Papp, Georg Anegg, and A. N. Yzelman. DAG scheduling in the BSP model. In *International Conference on Current Trends in Theory and Practice of Computer Science (SOFSEM)*, pages 238–253. Springer, 2025.

[PBY25] Pál András Papp, Toni Böhnlein, and A. N. Yzelman. Multiprocessor scheduling with memory constraints: Fundamental properties and finding optimal solutions. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP)*, page 238–247, 2025.

[PBY26] P. A. Papp, T. Böhnlein, and A. N. Yzelman. Replication in graph partitioning and scheduling problems. Submitted, 2026.

[Pet16] A. Petitet. A BSP scalability analysis of 3D LU factorization with row partial pivoting. SIAM Conference on Parallel Processing for Scientific Computing (PP16), April 2016.

[PGM+19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[PSFL20] Doru Thom Popovici, Martin D. Schatz, Franz Franchetti, and Tze Meng Low. A flexible framework for multidimensional DFTs. *SIAM Journal on Scientific Computing*, 42(5):C245–C264, 2020.

[PSSD14] Jongsoo Park, Mikhail Smelyanskiy, Narayanan Sundaram, and Pradeep Dubey. Sparsifying synchronization for high-performance shared-memory sparse triangular solver. In *Supercomputing: 29th International Conference, ISC 2014, Leipzig, Germany, June 22-26, 2014. Proceedings 29*, pages 124–140. Springer, 2014.

[PSSS21] Merten Popp, Sebastian Schlag, Christian Schulz, and Daniel Seemaier. Multilevel acyclic hypergraph partitioning. In *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 1–15, 2021.

[PSY25] Pál András Papp, Aleksandros Sobczyk, and A. N. Yzelman. The impact of partial computations on the red-blue pebble game. In *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 328–338, 2025.

[PVG+11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.

[PY88] Christos Papadimitriou and Mihalis Yannakakis. Towards an architecture-independent analysis of parallel algorithms. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC '88, page 510–513, New York, NY, USA, 1988. Association for Computing Machinery.

[RR52] J. Rutledge and H. Rubinstein. High order matrix computation on the UNIVAC. Presented at the meeting of the Association for Computing Machinery; Copies available from Sivan Toledo, May 1952.

[RS87] Victor J Rayward-Smith. UET scheduling with unit interprocessor communication delays. *Discrete Applied Mathematics*, 18(1):55–71, 1987.

[Sav95] John E Savage. Extending the Hong-Kung model to memory hierarchies. In *International Computing and Combinatorics Conference*, pages 270–281. Springer, 1995.

[SJZY23] Daniele G. Spampinato, Denis Jelovina, Jiawei Zhuang, and A. N. Yzelman. Towards structured algebraic programming. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming*, ARRAY 2023, pages 50–61, New York, NY, USA, 2023. Association for Computing Machinery.

[SLV18] Piyush Sao, Xiaoye Sherry Li, and Richard Vuduc. A communication-avoiding 3D LU factorization algorithm for sparse matrices. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 908–919. IEEE, 2018.

[SMHD13] Edgar Solomonik, Devin Matthews, Jeff Hammond, and James Demmel. Cyclops tensor framework: Reducing communication and eliminating load imbalance in massively parallel contractions. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 813–824. IEEE, 2013.

[Sob26] Aleksandros Sobczyk. I/O complexity and pebble games with partial computations. *Information Processing Letters*, page 106637, 2026.

[SY19] W. J. Suijlen and A.N. Yzelman. Lightweight Parallel Foundations: a model-compliant communication layer, 2019.

[SY24] Barna Saha and Christopher Ye. I/O complexity of attention, or how optimal is FlashAttention? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[Tis02] A. Tiskin. Bulk-synchronous parallel Gaussian elimination. *Journal of Mathematical Sciences*, 108(6):977–991, 2002.

[TKC19] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[Tol99] Sivan Toledo. A survey of out-of-core algorithms in numerical linear algebra. *External Memory Algorithms and Visualization*, 50:161–179, 1999.

[Val90] Leslie G. Valiant. A bridging model for parallel computation. *Commun. ACM*, 33(8):103–111, August 1990.

[Val11] Leslie G. Valiant. A bridging model for multi-core computing. *Journal of Computer and System Sciences*, 77(1):154–166, 2011.

[WCS+25] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, et al. Tilelang: A composable tiled programming model for AI systems. *arXiv preprint arXiv:2504.17577*, 2025.

[YDNNS20] A. N. Yzelman, D. Di Nardo, J. M. Nash, and W. J. Suijlen. A C++ GraphBLAS: specification, implementation, parallelisation, and evaluation, 2020.

[Yze22] A. N. Yzelman. Algebraic programming. https://algebraic-programming.github.io, 2022. Accessed: 2026-04-06.

[Yze24] A. N. Yzelman. Humble heroes. *Communications of Huawei Research*, 6:146–170, June 2024.

[ZCL+22] Behrooz Zarebavani, Kazem Cheshmi, Bangtian Liu, Michelle Mills Strout, and Maryam Mehri Dehnavi. HDagg: hybrid aggregation of loop-carried dependence iterations in sparse matrix computations. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1217–1227. IEEE, 2022.

[ZXW+16] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al. Apache Spark: a unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

(All authors) Computing Systems Lab, Huawei Research Zürich

URL, R. S. Steiner: https://raphaelssteiner.com/

URL, A. N. Yzelman: https://albert-jan.yzelman.net
Email address: albert-jan@yzelman.net